Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Lab 1: Python, NumPy and TensorFlow/Keras Environment

Objective:

To introduce students to numerical computing with NumPy arrays and matrix operations, data visualization with Matplotlib, and the fundamentals of tensors and neural network model building using TensorFlow/Keras, within the Google Colab environment.

Software and Tools:

1.6 GHz or faster processor
4 GB of RAM
20 GB of available hard disk space
Windows 7 or later
Google Colab

Colab Notebook for Lab 2:

https://colab.research.google.com/drive/1web3V6gP8QmyOGWXZUiXtvQ1Ious2Jxm?usp=sharing

Students should open the notebook, follow instructions, and perform the tasks to complete the lab exercises.

Background and Procedure:

Introduction

Before Artificial Neural Networks (ANNs) can be built, a student must be comfortable with the numerical tools that power them. Every operation inside a neural network, combining inputs with weights, applying an activation function, or updating parameters during training is, at its core, an array or matrix operation.

The goal of this lab is to build a strong foundation in NumPy arrays and matrix operations, learn to visualize data with Matplotlib, and get hands-on with TensorFlow tensors and the Keras API. By the end of this lab, students will be able to create and manipulate numerical data, visualize it clearly, and build and train a simple neural network model.

Environment Setup

1. In the first cell of your notebook, import numpy, matplotlib.pyplot, and tensorflow.
2. Print the version numbers of NumPy and TensorFlow to confirm the environment is ready.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("NumPy version:", np.__version__)
print("TensorFlow version:", tf.__version__)
```

```
NumPy version: 2.0.2
TensorFlow version: 2.20.0
```

NumPy Arrays for Numerical Computing

NumPy is the core Python library for numerical computing. Its main data structure, the ndarray (n-dimensional array), stores numbers of a single type in contiguous memory. This makes NumPy arrays far faster than ordinary Python lists for mathematical operations, because calculations are vectorized — applied to entire arrays at once instead of looping element by element.

Key NumPy concepts used in this lab:

- Array creation: np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()
- Shape and dtype: every array has a shape (its dimensions) and a dtype (element type)
- Indexing and slicing: accessing elements or sub-arrays with [ ]
- Reshaping: changing an array's dimensions without changing its data
- Vectorized operations: applying math to a whole array at once (a + b, a * 2, np.sqrt(a))
- Aggregations: sum(), mean(), max(), argmax()

Matrix Operations with NumPy

Matrices are two-dimensional arrays, and most of the computation inside a neural network combining input data with weight matrices, relies on matrix operations. NumPy's linear algebra module (np.linalg) provides these operations directly.

- Element-wise multiply (A * B) vs. true matrix multiply (A @ B)
- Transpose (A.T): flips rows and columns
- Determinant (np.linalg.det(A)) and inverse (np.linalg.inv(A))
- Eigenvalues and eigenvectors (np.linalg.eig(A))
- Solving a linear system A x = b with np.linalg.solve(A, b)

```
# Creating arrays
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])

print("1D array:", a)
print("2D array:\n", b)
print("Shape of a:", a.shape, "| Shape of b:", b.shape)
print("Dtype:", a.dtype)
```

```
1D array: [1 2 3 4 5]
2D array:
 [[1 2 3]
 [4 5 6]]
Shape of a: (5,) | Shape of b: (2, 3)
Dtype: int64
```

```
# Handy array creators
zeros = np.zeros((2, 3))
ones = np.ones((3, 2))
identity = np.eye(3)
arange = np.arange(0, 10, 2)          # start, stop, step
linspace = np.linspace(0, 1, 5)       # start, stop, num points
random_arr = np.random.rand(2, 2)     # uniform [0, 1)

print("zeros:\n", zeros)
print("ones:\n", ones)
print("identity:\n", identity)
print("arange:", arange)
print("linspace:", linspace)
print("random:\n", random_arr)
```

```
zeros:
 [[0. 0. 0.]
 [0. 0. 0.]]
ones:
 [[1. 1.]
 [1. 1.]
 [1. 1.]]
identity:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
arange: [0 2 4 6 8]
linspace: [0.   0.25 0.5  0.75 1.  ]
random:
 [[0.26556769 0.11684178]
 [0.35490657 0.99723134]]
```

Introduction to Tensors and TensorFlow

TensorFlow is a library for building and training machine learning models. Its core data structure, the tensor, generalizes NumPy arrays: tensors can hold data of any number of dimensions, can run on a GPU/TPU for speed, and can automatically track the operations performed on them to compute gradients the mechanism neural networks use to learn.

- tf.constant() creates a fixed tensor; tf.Variable() creates one that can change (used for trainable weights)
- Tensors support the same kind of operations as NumPy arrays (matmul, transpose, reduce_sum, etc.)
- tf.GradientTape() records operations so TensorFlow can automatically compute derivatives (gradients)

```
# Creating tensors
t1 = tf.constant([1, 2, 3])
t2 = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
t_zeros = tf.zeros((2, 3))
t_ones = tf.ones((3, 2))
t_range = tf.range(10)

print("t1:", t1)
print("t2:\n", t2)
print("Shape:", t2.shape, "| Dtype:", t2.dtype)

# Convert between NumPy and TensorFlow freely
np_from_tensor = t2.numpy()
tensor_from_np = tf.convert_to_tensor(np.array([5, 6, 7]))
print("\nNumPy from tensor:\n", np_from_tensor)
print("Tensor from NumPy:", tensor_from_np)
```

```
t1: tf.Tensor([1 2 3], shape=(3,), dtype=int32)
t2:
 tf.Tensor(
[[1. 2.]
 [3. 4.]], shape=(2, 2), dtype=float32)
Shape: (2, 2) | Dtype: <dtype: 'float32'>

NumPy from tensor:
 [[1. 2.]
 [3. 4.]]
Tensor from NumPy: tf.Tensor([5 6 7], shape=(3,), dtype=int64)
```

```
# Tensor operations mirror NumPy
t_a = tf.constant([[1., 2.], [3., 4.]])
t_b = tf.constant([[5., 6.], [7., 8.]])

print("Element-wise add:\n", t_a + t_b)
print("\nMatrix multiply:\n", tf.matmul(t_a, t_b))
print("\nTranspose:\n", tf.transpose(t_a))
print("\nReduce sum:", tf.reduce_sum(t_a))
print("Reduce mean along axis 0:", tf.reduce_mean(t_a, axis=0))
```

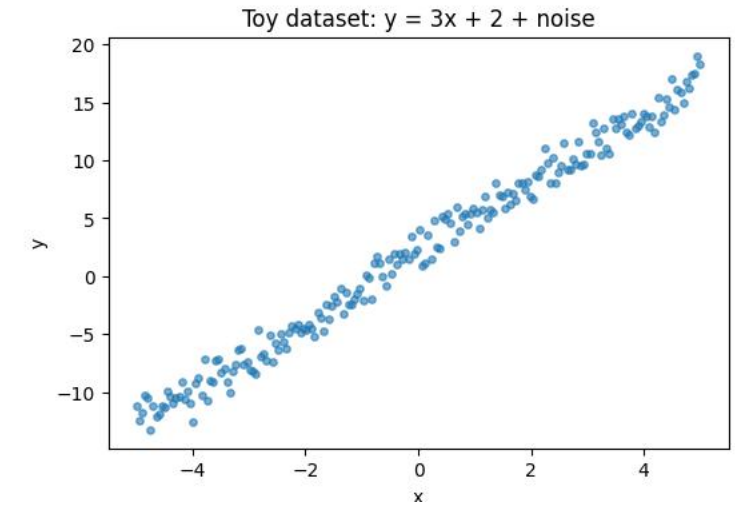
```
Element-wise add:
 tf.Tensor(
[[ 6.  8.]
 [10. 12.]], shape=(2, 2), dtype=float32)

Matrix multiply:
 tf.Tensor(
[[19. 22.]
 [43. 50.]], shape=(2, 2), dtype=float32)

Transpose:
 tf.Tensor(
[[1. 3.]
 [2. 4.]], shape=(2, 2), dtype=float32)

Reduce sum: tf.Tensor(10.0, shape=(), dtype=float32)
Reduce mean along axis 0: tf.Tensor([2. 3.], shape=(2,), dtype=float32)
```

Building Neural Networks with Keras

Keras is TensorFlow's high-level API for building neural networks. Using the Sequential API, layers are stacked in order, the model is compiled with an optimizer and a loss function, and then trained (fit) on data.

- Build: tf.keras.Sequential([...]) stacks layers such as Dense
- Compile: model.compile(optimizer, loss) configures how the model learns
- Train: model.fit(X, y, epochs=n) adjusts the model's weights to reduce the loss
- Predict: model.predict(X_new) generates outputs for new inputs

```
# Generate a toy regression dataset: y = 3x + 2 + noise
np.random.seed(0)
X = np.linspace(-5, 5, 200).reshape(-1, 1).astype("float32")
y_true = 3 * X + 2 + np.random.normal(0, 1, X.shape).astype("float32")

plt.figure(figsize=(6, 4))
plt.scatter(X, y_true, alpha=0.6, s=15)
plt.title("Toy dataset: y = 3x + 2 + noise")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



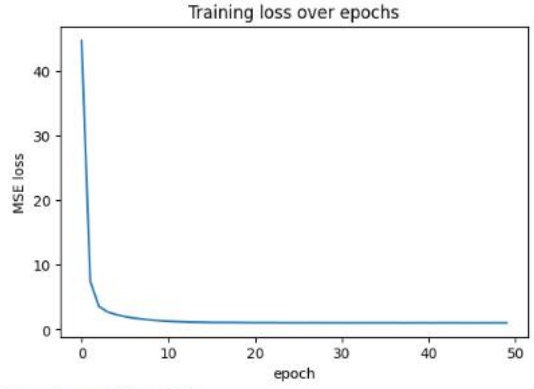
```
# Build a simple Sequential model (one dense layer = linear regression)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=(1,))
])

model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
              loss="mse")

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	2

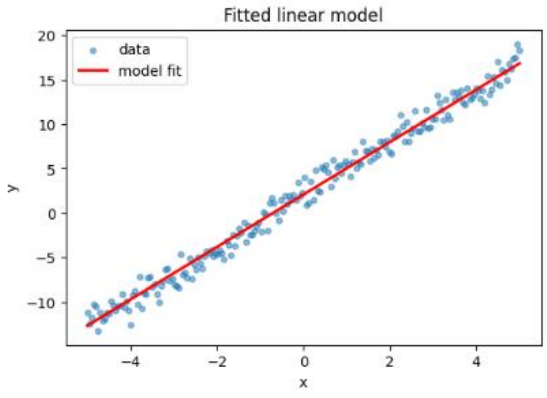
```
Total params: 2 (8.00 B)
Trainable params: 2 (8.00 B)
Non-trainable params: 0 (0.00 B)
```

```
# Train the model
history = model.fit(X, y_true, epochs=50, verbose=0)

plt.figure(figsize=(6, 4))
plt.plot(history.history["loss"])
plt.title("Training loss over epochs")
plt.xlabel("epoch")
plt.ylabel("MSE loss")
plt.show()

learned_weight, learned_bias = model.layers[0].get_weights()
print(f"Learned: y = {learned_weight[0][0]:.2f}x + {learned_bias[0]:.2f}")
print("True relationship: y = 3x + 2")
```




```
Learned: y = 2.95x + 2.08
True relationship: y = 3x + 2
```

```
# Visualize the fitted line against the data
y_pred = model.predict(X, verbose=0)

plt.figure(figsize=(6, 4))
plt.scatter(X, y_true, alpha=0.5, s=15, label="data")
plt.plot(X, y_pred, color="red", linewidth=2, label="model fit")
plt.title("Fitted linear model")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```



Submission:

- Submit the Colab notebook with all cells executed and outputs visible.
- Ensure the notebook is well-commented and demonstrates your understanding of each operation.